A feasibility-aware predict-then-optimize framework for operational MILPs

Javier Cornejo[1], Sebastián Dávila-Gálvez[1,2], Franco Quezada[1,2] and Luis Rojo-González[1,2,3,4]

[1]Department of Industrial Engineering, Universidad de Santiago de Chile, Av. Ecuador 3769, Estación Central, Santiago, 9170124, Chile

[2]Program for the Development of Sustainable Production Systems (PDSPS), Av. Ecuador 3769, Estación Central, Santiago, 9170124, Chile

[3]Centre Interuniversitaire de Recherche sur les Réseaux d'Entreprise, la Logistique et le Transport (CIRRELT) and Département de Mathématiques et de Génie industriel, Polytechnique Montréal, Montréal, H3C 3A7, Quebec, Canada

[4]Univ. Lille, Inria, CNRS, Centrale Lille, UMR 9189 CRIStAL, Lille, F-59000, France

March 17, 2026

Abstract

Combinatorial optimization problems present significant computational challenges due to nonconvex feasible regions and exponentially growing solution spaces. In this paper, we investigate a predict–then–optimize framework that integrates supervised ML techniques including K–Nearest Neighbors, Decision Trees, Naïve Bayes, and Support Vector Machines with mixed integer linear programming to solve combinatorial optimization problems. The framework is tested on the Multi-item Capacitated Lot-Sizing Problem (MCLSP). Inspired by the Phase I simplex method, a discrimination rule defines the estimation to ensure a feasible solution. Extensive computational experiments reveal that using strengthen linear relaxation features to train ML models can reduce resolution times by up to 99% while maintaining optimality gaps below 1%. These results underscore the potential of combining ML and combinatorial optimization to achieve scalable, near-optimal solutions in time-sensitive decision-making contexts.
Keywords: Predict-then-optimize, Supervised machine learning, Mixed-integer linear programming, Capacitated lot-sizing problem..

1 Introduction

Combinatorial optimization (CO) problems, especially those formulated as mixed-integer linear programming (MILP models) are inherently challenging due to the nonconvex and discrete nature of their feasible regions. Unlike linear programming, where feasible solutions can be efficiently explored within a convex set, MILPs typically require implicit enumeration of feasible discrete solutions, often through branch-and-bound algorithms (Hoffman, 2000). As a result, solving MILPs becomes computationally expensive, especially for large-scale instances (Bertsimas and Weismantel, 2005). While exact solutions may be viable at the strategic or tactical level, they are often impractical for operational decision-making, where rapid solutions are required.

A wide range of techniques have been proposed to address this challenge, including exact methods such as Benders decomposition (Rahmaniani et al., 2017), Dantzig-Wolfe decomposition (Desaulniers, Desrosiers, and Solomon, 2006), and cutting-plane methods (Cornuéjols, 2008), which are often combined into branch-and-cut frameworks. When exact methods are computationally impracticable, heuristics provide a practical alternative for generating high-quality solutions. These include both classical heuristics based on intuitive decision rules (Mart, Pardalos, and Resende, 2018) and metaheuristics such as simulated annealing, tabu search, and ant colony optimization (Desale et al., 2015; Reeves, 1996), which explore the solution space more systematically. A further evolution involves hyper-heuristics, which dynamically select among a pool of heuristics based on performance (Burke et al., 2019). More recently, matheuristics, which are hybrid approaches that embed mathematical programming within metaheuristic frameworks, have been developed

and successfully applied to problems in routing (Archetti and Speranza, 2014), lot-sizing (Cunha, Kramer, and Melo, 2019), clustering (Gnägi and Baumann, 2021), and scheduling (Zhang et al., 2022), among others.

Recently, machine learning (ML) models have gained attention as solution methods for combinatorial optimization models. ML models can be employed either to directly predict solutions or to assist exact methods, such as guiding the branching decisions in branch-and-bound algorithms. Vanderschueren et al. (2022) categorize these strategies into two paradigms: predict-then-optimize (sequential) and predict-and-optimize (integrated). Both approaches aim to reduce the computational effort required to solve large-scale optimization problems. In particular, the predict-then-optimize strategy offers additional flexibility, as it enables repeated optimization based on previously solved instances employing imitation learning (Hussein et al., 2017).

Figure 1 illustrates the general workflow of the predict-then-optimize framework using imitation learning. In this framework, an ML model is trained on a set of previously solved instances to imitate the behavior of an optimization solver. Once trained, the ML model is used to make predictions for new instances, subsequently refined by solving a restricted optimization problem.

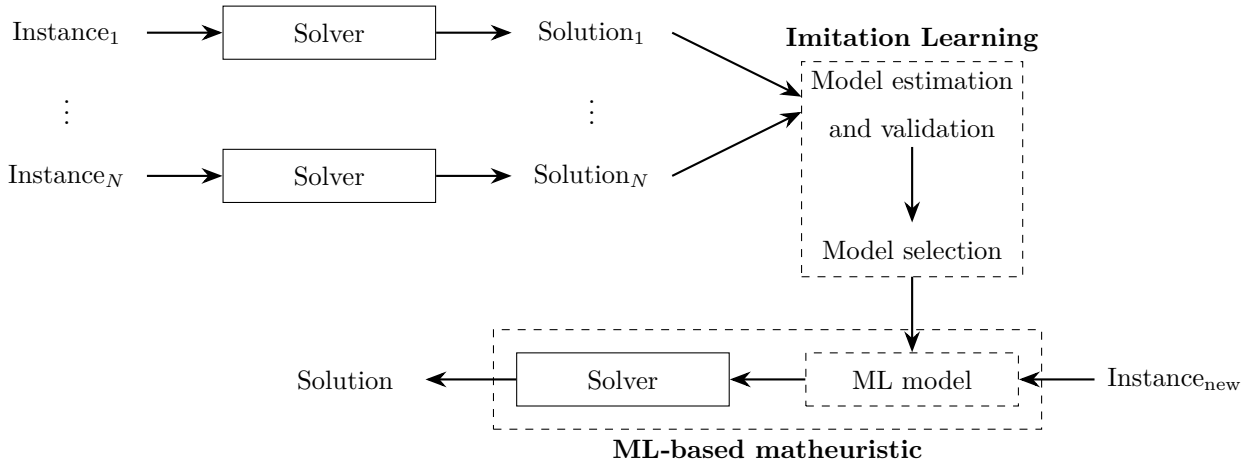


Figure 1: Illustration of a general predict-then-optimize framework using imitation learning.

A critical requirement in this context is access to a dataset of optimally solved instances for training. While deep learning models are frequently employed due to their predictive accuracy (Sultana et al., 2022; Yilmaz and Büyüktahtakın, 2022, 2024), they typically require large training datasets, significant computational effort during training, and suffer from limited interpretability (Fan et al., 2021; Tsimenidis, 2020). These demands pose significant barriers to real-world applications, particularly in settings with constrained computational infrastructure.

Moreover, ML predictions may result in infeasible solutions when fixed in the optimization model. Additionally, prediction accuracy can degrade when test instances differ significantly from the training data in

size (scalability) or distribution (data drift) (Bengio, Lodi, and Prouvost, 2021).

In this work, we focus on classical ML models that can be trained on relatively small datasets to develop predict-then-optimize matheuristics for MILPs with binary variables, ensuring feasibility. Our framework includes a discrimination rule that rescues feasibility, which is based on the Phase I simplex method. We also evaluate the robustness of our method with respect to scalability and data shift, using datasets of varying sizes and distributions.

The remainder of the paper is structured as follows. Section 2 reviews relevant literature on the integration of machine learning into combinatorial optimization, highlighting the contribution of this work. Section 3 formalizes the proposed predict-then-optimize framework for mixed-integer linear programs (MILPs) with binary variables. Section 4 presents comprehensive computational experiments using a case study to evaluate the performance of the proposed approach. Finally, Section 5 summarizes the main findings and discusses their practical implications.

2 Related works

Imitation learning as a framework to solve CO is a rapidly evolving field that leverages the predictive power of ML models to enhance efficiency to improve the performance of resolution methods. The literature identifies three key approaches in this scope: (i) End-to-End Learning, (ii) Learning to Configure Algorithms, and (iii) Learning Alongside Optimization Algorithms (Bengio, Lodi, and Prouvost, 2021). First, *End-to-End Learning* employs ML models to directly generate solutions for discrete optimization problems based on the input instance. Examples of this approach include ML-based heuristics that predict high-quality solutions based on features derived from problem instances (see e.g., Bello et al., 2016; Kotary et al., 2021; Larsen et al., 2022). Second, *Learning to Configure Algorithms* focuses on identifying optimal parameter settings for CO algorithms. These algorithms often require predefined parameters that remain fixed throughout the optimization process, which can significantly impact the performance. ML can assist in selecting these parameters, such as determining the aggressiveness of pre-solving operations in a MILP solver or tuning the step size in gradient descent methods for convex optimization (see e.g., Bischl et al., 2016; Chen et al., 2022; Hutter et al., 2014). Finally, *Learning Alongside Optimization Algorithms* integrates ML models into the iterative process of a CO algorithm. Unlike previous approaches, which provide one-time predictions or parameter configurations using ML, this method repeatedly utilizes the same ML model to guide decision-making throughout the optimization process. For instance, in the branch-and-bound algorithm for MILP problems, ML models can dynamically select branching variables or prioritize nodes for exploration (see e.g., (Khalil et al., 2016; Khalil et al., 2017; Nair et al., 2020)).

Although deep learning models exhibit strong predictive performance, their effectiveness heavily relies on access to large-scale training datasets. In this context, this requires solving a large number of instances to

optimality beforehand, which can be computationally expensive and time-consuming. Furthermore, training deep learning models typically incurs significant computational costs due to their complex, multi-layered architectures and the large number of parameters involved (Fan et al., 2021; Tsimenidis, 2020).

In addition, the estimation proposed for ML or deep learning in the context of CO, introduces three well-known challenges (Bengio, Lodi, and Prouvost, 2021): *feasibility*, ensuring that predictions always satisfy all problem constraints; *scalability*, maintaining the model's performance when tested on larger instances than those obtained during training; and *data drift*, the degradation in performance that arises when the data distribution shifts between training and testing in new instances.

Feasibility is not always ensured. For instance, the approach presented in Lin et al. (2019) applied decision tree (DT) and random forest (RF) regression models to approximate MILP solutions for the security constrained unit commitment (SCUC) problem by first solving the linear relaxation and then predicting the continuous residual value between the binary and its linear relaxation. They added the predicted residuals to the relaxed binary value and rounded the result to obtain a candidate integer solution. However, this prediction could violate SCUC capacity constraints. Similarly, Xavier, Qiu, and Ahmed (2019) utilized K-nearest neighbor (KNN) and support vector machines (SVM) to enhance MILP solvers, testing their framework on SCUC problem, by predicting redundant constraints and generating warm starts with KNN, while SVM is trained to predict affine subspaces where the optimal solution is likely to reside, aiming to reduce feasible region. However, predicting affine subspaces can lead to infeasible solutions as shown in computational experiments.

By contrast, works such as Jia and Shen (2019), Morabit, Desaulniers, and Lodi (2023) and Alvarez, Louveaux, and Wehenkel (2014) leveraged methods like Benders decomposition or column generation, which inherently maintain feasibility. For example, Jia and Shen (2019) train an SVM classifier to identify valuable Benders cuts to accelerate Benders decomposition for two-stage stochastic programs. Similarly, Morabit, Desaulniers, and Lodi (2023) proposed an ML approach for arc selection in column generation applied to constrained shortest path problems. They tested their method on the vehicle and crew scheduling problem (VCSP) and the vehicle routing problem with time windows (VRPTW), both of which involve a shortest path problem with resource constraints as the pricing subproblem. Finally, Alvarez, Louveaux, and Wehenkel (2014) trained Extremely Randomized Trees to imitate effective branching strategies in the branch-and-bound algorithm. Additionally, some optimization problems are structured so that infeasible predictions cannot arise. For example Rachelson, Abbes, and Diemer (2010) integrated MILP with boosting using weak classifiers (DT and SVM) for intra-daily electricity planning problem (PP), where unmet demand incurs a penalty cost rather than violating constraints.

Although our work focuses on ML rather than deep learning, feasibility-repair mechanisms from deep learning approaches can be adapted to our framework. For example, Morabit, Desaulniers, and Lodi (2022)

trained an artificial neural network (ANN) to predict the probability that each edge belongs to an optimal solution of the capacitated vehicle routing problem (CVRP). Since fixing high probability edges may produce routes whose total demand exceeds vehicle capacity, they iteratively "unfix" the lowest probability edge on any infeasible route until capacity constraints are met, thereby preserving as many likely edges as possible. In a similar way, Yilmaz and Büyüktahtakın (2022) proposed a relational neural network for predicting binary decisions in the multi-item capacitated lot sizing problem (MCLSP). This approach was extended in Yilmaz and Büyüktahtakın (2024) by integrating active constraint prediction reducing problem size and applying the same variable unfixing repair strategy from Morabit, Desaulniers, and Lodi (2022) to restore feasibility. However, feasibility repair presents two drawbacks. First, ML models selected based on validation metrics such as classification accuracy of non-proven feasible predictions may be inconsistent when applied to the optimization model, potentially leading to paradoxical results where a model with higher accuracy produces worse outcomes after the feasibility check. Second, the number of iterations required to find a feasible solution depends on the step size, introducing computational cost. While Yilmaz and Büyüktahtakın (2024) addressed this by predicting tight constraints on a relaxed version of the problem, this requires training an additional model and may not guarantee computational efficiency gains.

On the other hand, **Scalability** is evaluated (e.g., (Jia and Shen, 2019), (Morabit, Desaulniers, and Lodi, 2023) and (Xavier, Qiu, and Ahmed, 2019)) by testing on datasets larger than those used for training, without explicitly proposing mechanisms to ensure scalable performance. Similarly, **data drift** is examined (e.g., (Jia and Shen, 2019) and (Xavier, Qiu, and Ahmed, 2019) through experiments under different distributions, but neither work presents explicit strategies to manage drift or scalability.

Table 1 summarizes the main characteristics and challenges of related works that combine machine-learning techniques with combinatorial optimization, emphasising classical ML models because of their lower data requirements.

2.1 Our contributions

We focus on supervised machine learning models that can be trained on relatively small datasets, thereby reducing the number of MILP instances that must be solved for training and lowering computational infrastructure requirements compared to deep learning approaches.

We also ensure the feasibility of our end-to-end predict-then-optimize framework through a discrimination rule based on the Phase I simplex method, which produces feasible binary predictions. We demonstrate the scalability of our approach by evaluating its performance across MCLSP instances of varying sizes. In addition, we assess robustness to distributional shifts by evaluating performance on instances drawn from distributions different from the training data.

Our computational experiments demonstrate competitive performance compared to baseline neural network-

Table 1: Summary of related works.

Reference	Strategy✣			Model	Challenges		
	i)	ii)	iii)		Feasibility	Scalability	Data drift
Rachelson, Abbes, and Diemer, 2010	✓			DT, SVM			
Alvarez, Louveaux, and Wehenkel, 2014			✓	DT			
Lin et al., 2019	✓			DT			
Jia and Shen, 2019			✓	SVM		✓	✓
Xavier, Qiu, and Ahmed, 2019				KNN,SVM		✓	✓
Yilmaz and Büyüktahtakın, 2022	✓			ANN		✓	✓
Morabit, Desaulniers, and Lodi, 2022	✓			ANN		✓	✓
Morabit, Desaulniers, and Lodi, 2023			✓	DT,ANN		✓	
Yilmaz and Büyüktahtakın, 2024	✓			ANN	✓	✓	✓
This work	✓			KNN, DT, NB, SVM	✓	✓	✓

✣ i) End-to-End - ii) Algorithm Configuration - iii) Alongside

based heuristics, both in terms of solution quality and computational efficiency.

3 Predict-then-optimize

In this section, a formal definition of mixed-integer linear programs is provided, which are the subject of this study. We then show an overview of the predict-then-optimize framework, including the process employed for the train.

3.1 Problem formulation

We study the following mixed-integer linear program (MILP):

$$\min_{\boldsymbol{x},\boldsymbol{y}} \quad c_1^\top x + c_2^\top y \tag{1.1}$$

$$\text{s.t.} \quad A_1 x + B_1 y \leq b_1. \tag{1.2}$$

$$x \in \mathbb{R}^{n_1}, y \in \{0,1\}^{n_2} \tag{1.3}$$

where n_1 and n_2 are the numbers of continuous and binary variables, respectively. We assume A_1, A_2, b_1, c_1, and c_2 are given rational matrices/vectors of appropriate size.

Note that model 1 describes, for instance, the optimization models where fixed costs are incurred only when the continuous variables take positive values. Such optimization models arise in various applications, including the lot-sizing problem, where a setup cost is incurred only when production occurs, and the p-

median problem, where the fixed cost is incurred only when the facility is located (e.g. see (Karimi, Fatemi Ghomi, and Wilson, 2003), (Jans and and, 2008), (Daskin and Maass, 2015)).

3.2 Imitation learning

In many practical applications, the same optimization problem must be solved repeatedly under varying input parameters, allowing the collection of a dataset of solved instances. We hypothesize that consistent patterns in the assignment of binary decision variables across these instances can be more effectively learned through supervised ML techniques than by relying solely on traditional heuristic methods.

One promising method that leverages this idea is *imitation learning*. According to Hussein et al. (2017), imitation learning refers to the process by which an agent acquires skills or behaviors by observing a teacher performing a task. In our context, the agent corresponds to the ML model, the teacher is the optimization solver, and the task is the assignment of binary decision variables in the solution of the problem. The goal is to train the ML model to replicate the solver's behavior when faced with new instances.

Let $\mathcal{D} = \{(H^i, y^{*i})\}_{i=1}^{I}$ denote a dataset of I solved instances, where each pair (H^i, y^{*i}) includes a feature set $H^i \in \mathcal{H} \subseteq \mathbb{R}^{d_h}$ (e.g., instance parameters, linear relaxation outputs) and the corresponding optimal binary solution y^{*i}. Also, let Λ be a set of different ML models. The goal is to estimate an ML model m_λ that, given an input H^i predicts the optimal binary decision vector y^{*i}. Since most supervised ML models produce a continuous output z, a post-processing step is required to obtain a binary decision $\hat{y} \in \{0,1\}^{n_2}$. For this purpose, we adopt the discrimination rule formalized in the next definition 1.

Definition 1 (Discrimination rule)**.** Let $z \in [0,1]^{n_2}$ be the continuous output vector of a ML model m_λ. A *discrimination rule* assigns to each value z a binary value $\hat{y}$, yielding a binary prediction vector $\hat{y} \in \{0,1\}^{n_2}$. A traditional discrimination rule $F_\beta(\boldsymbol{z})$ can be defined as $\hat{y} = 1$ if $z \geq \beta$, and $\hat{y} = 0$ otherwise, where $\beta \in [0,1]$ is an arbitrary discrimination threshold.

Based on the traditional discrimination rule and the Phase I method of the Simplex algorithm, an optimization model is proposed in order to obtain feasible solutions. The Phase I method is a classical procedure in linear programming designed to determine the feasibility of a linear system $Ax \leq b$. It introduces slack variables s to construct an auxiliary linear program $\{\min \mathbf{1}^\top s : Ax + s \leq b\}$ whose objective is to minimize the total infeasibility. If the optimal value of this auxiliary problem is zero, the original model is feasible, and its feasible solution can be used to initialize the original problem. Otherwise, the system is declared infeasible (Bertsimas and Tsitsiklis, 1997; Vanderbei, 2020).

Thus, we propose an optimization-based discrimination rule that incorporates the ML prediction vector z and a classification threshold β to generate a binary decision vector $\hat{y}$ that satisfies feasibility, which is

described as follows:

$$F_\beta(\boldsymbol{z}) := \min_{\boldsymbol{x},\boldsymbol{y},\boldsymbol{u},\boldsymbol{v}} \quad \sum_{p\in P_1}(u_p + (1-z_p)u_p) + \sum_{p\in P_2}(v_p + z_p v_p) \tag{2.1}$$

$$\text{s.t.} \quad A_1 x + B_1 \hat{y} \leq b_1, \tag{2.2}$$

$$\hat{y}_p \geq (z_p - \beta) - v_p \qquad \forall p \in P_1 := \{p \in P \mid z_p \geq \beta\} \tag{2.3}$$

$$\hat{y}_p \leq 1 + (z_p - \beta) + u_p \qquad \forall p \in P_2 := \{p \in P \mid z_p < \beta\} \tag{2.4}$$

$$\hat{y}_p \in \{0,1\}, \quad u_p, v_p \in \{0,1\}, \quad x \in \mathbb{R}^{n_2} \qquad \forall p \in \{1,\ldots,n_2\} \tag{2.5}$$

If the original prediction $\hat{y}$ leads to an infeasible solution, the optimization model activates one or more slack variables to flip specific entries of $\hat{y}$ and recover feasibility. In particular, setting $v_p = 1$ allows the model to flip $\hat{y}_p$ from 1 to 0, while setting $u_p = 1$ flips $\hat{y}_p$ from 0 to 1. This mechanism enables a minimal adjustment to the original prediction, preserving as much of the ML model's output as possible while ensuring a feasible solution to the MILP.

Constraints (2.3) and (2.4) encode the threshold-based discrimination logic of Definition 1. If the prediction $\hat{y}$ obtained at threshold β is already feasible, the optimal objective value of the auxiliary model is zero, and all slack variables remain inactive. Conversely, if feasibility is not achieved, the model activates one or more slack variables to flip the values of selected components of $\hat{y}$. More precisely, if $v_p = 1$, then $\hat{y}_p$ is flipped from 1 to 0; analogously, if $u_p = 1$, then $\hat{y}_p$ is flipped from 0 to 1. The objective function assigns a cost to each flip using two components: (i) a unit penalty for each activated slack variable u_p or v_p, and (ii) a continuous confidence-weighted penalty based on the ML output z. Specifically, for the variable $\{u_p\}_{p\in P_1}$ the penalty term is $(1 - z_p)$; while for the variable $\{v_p\}_{p\in P_2}$ the penalty is z_p. This formulation ensures that the total number of flips is minimized while favoring changes to predictions with lower confidence, thereby preserving classification fidelity.

Finally, to obtain a complete feasible solution $(\hat{x}, \hat{y})$ to the MILP model, the binary estimation value $\hat{y}$, obtained by solving the classification model $F_\beta(\boldsymbol{z})$ is fixed as a constraint in the model 1 (i.e., $y = \hat{y}$). The MILP($\hat{y}$) is then solved to determine the corresponding continuous variables $\hat{x}$.

3.2.1 Model estimation, validation and selection

In order to predict the binary decision variables $\boldsymbol{y}^{*i}$, the dataset $\mathcal{D}$ is first partitioned into training and test sets, as formalized in the following definition.

Definition 2 (Train and test instances)**.** Let $\mathcal{D}$ be the set of solved instances such that $\mathcal{D} := \mathcal{D}_1 \cup \mathcal{D}_2$, where $\mathcal{D}_1$ and $\mathcal{D}_2$ are the sets of instances in the train and test sets, respectively. Let $\alpha_1 \in (0,1)$ be the portion size of $\mathcal{D}_1$ with respect to $\mathcal{D}$, i.e., $|\mathcal{D}_1| = \alpha_1|\mathcal{D}|$ and $|\mathcal{D}_2| = (1-\alpha_1)|\mathcal{D}|$.

The training dataset is used to estimate the model hyperparameters, while the test dataset serves to evaluate the model's generalization performance on unseen instances.

Hyperparameter selection is performed after the dataset is split. While hyperparameter tuning could be formulated as an optimization problem, it is generally non-convex and computationally expensive to solve directly (Feurer and Hutter, 2019). Therefore, in this work, we adopt a grid search strategy, which involves evaluating a predefined set of hyperparameter combinations. In addition, to robustly assess the performance of each combination and to mitigate overfitting, we employ k-fold cross-validation (Hastie et al., 2005), defined as follows.

Definition 3 (Cross definition)**.** The ML model obtained relies on the input data, where the training set $\mathcal{D}_1$ is further partitioned into two subsets. Let $\alpha_2 \in (0, 1)$ denote portion size of the training fold $\mathcal{D}_1^\alpha$ and validation fold $\mathcal{D}_1^\beta$ with respect to train set $\mathcal{D}_1$, i.e., $|\mathcal{D}_1^\alpha| = \alpha_2|\mathcal{D}_1|$ and $|\mathcal{D}_1^\beta| = (1-\alpha_2)|\mathcal{D}_1|$ respectively. More precisely, $\mathcal{D}_1 = \mathcal{D}_1^\alpha \cup \mathcal{D}_1^\beta$ can be shuffled in several ways to generate different pairs of training and validation folds. Let Ω be the number of combinations to shuffle $\mathcal{D}_1$. For each $\omega \in \Omega$, let $\mathcal{D}_{1,\omega}^\alpha$ and $\mathcal{D}_{1,\omega}^\beta$ represent specific folds used for model estimation and performance evaluation, respectively.

As shown in Definition 3, cross-validation involves a repeated train of an ML model applied to several folds, and the resulting models are subsequently tested on other subsets (validation folds), that were not used during training (Berrar, 2019). Moreover, a leave-one-out cross-validation scheme was applied ensuring that, in each shuffle $w \in \Omega$, the instances contained in the train set $\mathcal{D}_1$ serve exactly once as validation folds $\mathcal{D}_{1,\omega}^\beta$, while the remaining instances form the training fold $\mathcal{D}_{1,\omega}^\alpha$, as illustrated in Figure 2.

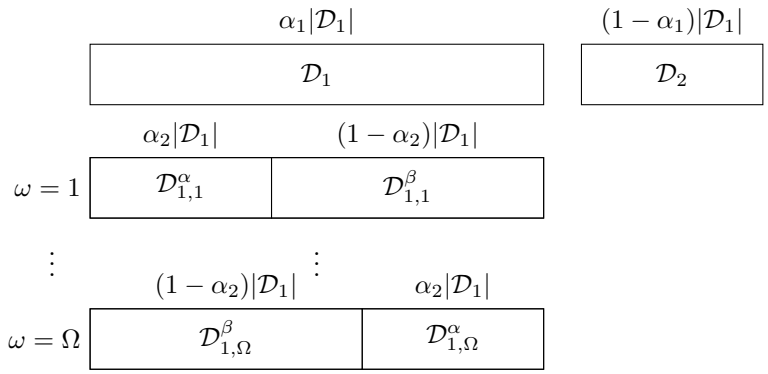


Figure 2: Cross validation scheme. Train dataset $\mathcal{D}_1$ is first split into $\mathcal{D}_1^\alpha$ and $\mathcal{D}_1^\beta$. For each of the Ω shuffles, $\mathcal{D}_1$ yields a fold w in which the training subset $\mathcal{D}_{1,\omega}^\alpha$ and validation subset $\mathcal{D}_{1,\omega}^\beta$ are formed.

Then, the performance on each validation fold $\mathcal{D}_1^\beta$ is measured using accuracy. Let $\hat{y}_w$ be the predictions from validation fold $\mathcal{D}_{1,\omega}^\beta$ obtained by the ML model trained with $\mathcal{D}_{1,\omega}^\alpha$, and y_w^* be the true values, accuracy can be defined as $\text{Acc}_w = \mathbb{1}_{\{\hat{y}_w = y_w^*\}}/|\mathcal{D}_{1,\omega}^\beta|$.

Once all combinations of hyperparameters are evaluated, the combination that yields the lowest coefficient of variation (CV) across the validation folds is selected. The CV is a standardized measure of dispersion, defined as the ratio of the standard deviation σ_{Acc} to the mean validation accuracy μ_{Acc}, i.e., $\text{CV} = \sigma_{\text{Acc}}/\mu_{\text{Acc}}$.

This criterion promotes the selection of hyperparameters that yield both high and stable predictive performance across folds, effectively balancing the bias-variance trade-off (Doroudi, 2020).

Finally, once the best hyperparameters are selected, the test dataset $\mathcal{D}_2$ is used to assess the potential overfitting of each ML model m_Λ through its accuracy, denoted as $\text{ACC}_{\mathcal{D}_2}$.

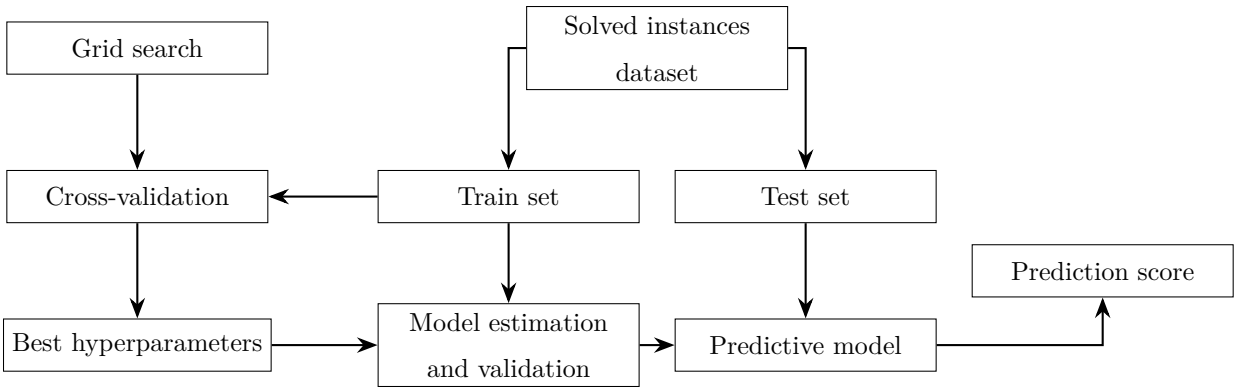


Figure 3: Process for train classification ML models.

The entire process is shown in Figure 3, while the evaluated hyperparameters are listed in Appendix A. It is important to highlight that, each time the models are trained, the training data is standardized, and the same transformation is applied to the corresponding test and validation sets. Specifically, the mean and standard deviation computed from the training set are used to normalize the other sets, ensuring consistency.

4 Computational experiments

This section presents computational experiments to evaluate the performance of the proposed predict-then-optimize framework. We first describe the multi-item capacitated lot-sizing problem (MCLSP), which serves as the test set for matheuristic, and provide details on the instance generation procedure. We then outline the experimental setup, including model estimation, validation, and selection (see 3.2.1) as well as the metrics used to assess computational efficiency and solution quality. Finally, we analyze the results across varying instance sizes and complexities, highlighting the improvements in runtime and the ability to obtain near-optimal solutions.

The framework was implemented using Python v3.10.12 and Gurobi v11.0.0 on a machine running Ubuntu 22.04.4 LTS with an Intel i7-12700 CPU, 32 GB of RAM, and an Intel AlderLake-S GT1 GPU 4.9 Ghz. ML models and techniques used in this work are implemented using *scikit-learn* library.

4.1 Multi-item Capacitated Lot-Sizing Problem

To evaluate our framework, we consider the multi-item capacitated lot-sizing problem. This problem involves determining the optimal production quantities in each time period for multiple items over a finite planning horizon in order to minimize total costs. The planning horizon $\mathcal{T} := \{1, \ldots, T\}$ represents the set of time periods, and $\mathcal{J} := \{1, \ldots, J\}$ represents the number of items. The production capacity in period t is given by c_t. The parameter p_t^j is the unit cost of production of item j during period t, h_t^j is the inventory holding cost of item j at the end of period t, f_t^j is the fixed setup cost that must be paid before any production of item j can occur in period t and the demand for item j in period t is d_t^j. The nonnegative variable X_t^j represents the quantity of item j produced in period t. The variable S_t^j represents stock or inventory of item j held over at the end of period t, and its non-negativity ensures that backorders are not permitted. Y_t^j indicates whether or not produce item j in period t. A MILP formulation of the MCLSP is defined as follows:

$$\min \quad \sum_{j\in\mathcal{J}}\sum_{t\in\mathcal{T}} h_t^j S_t^j + f_t^j Y_t^j + p_t^j X_t^j \tag{3.1}$$

$$\text{s.t.} \quad S_t^j = S_{t-1}^j + X_t^j - d_t^j \qquad \forall t \in \mathcal{T}, j \in \mathcal{J} \tag{3.2}$$

$$X_t^j \le c_t Y_t^j \qquad \forall t \in \mathcal{T}, j \in \mathcal{J} \tag{3.3}$$

$$\sum_{j\in\mathcal{J}} X_t^j \le c_t \qquad \forall t \in \mathcal{T} \tag{3.4}$$

$$Y_t^j \in \{0,1\} \qquad \forall t \in \mathcal{T}, j \in \mathcal{J} \tag{3.5}$$

$$X_t^j, S_t^j \ge 0 \qquad \forall t \in \mathcal{T}, j \in \mathcal{J} \tag{3.6}$$

The objective of the MCLSP is to minimize the total cost, which includes production costs, inventory holding costs, and fixed setup costs. Constraints (3.2) express the inventory balance. Due to restrictions (3.3), production of an item can only take place if the resource is setup for that particular item. Constraints (3.4) are the limited quantity to be produced. The set of constraints (3.2) and (3.6) ensure that demand for each item is fulfilled without backlogging. Finally, constraints (3.5) and (3.6) define the domains of the binary and continuous decision variables, respectively.

Note that when the binary variable vector $\mathbf{Y}$ is fixed with predictions $\hat{y}$, the MCLSP instance may become infeasible if the demand in certain periods cannot be met due to insufficient capacity in previously active production periods. In such cases, infeasibility arises from periods that are inactive ($\hat{y}_t^j = 0$) rather than overproduction ($\hat{y}_t^j = 1$). Consequently, in discrimination rule $F_\beta(\boldsymbol{z})$ the slack variable $\boldsymbol{v}$, which controls the flipping of predicted values from 1 to 0, can be fixed at zero.

Furthermore, to avoid incurring unnecessary setup costs, the predicted vector $\hat{y}$ generated by the discrimination rule can be post-processed based on the solution of the restricted problem MCLSP($\hat{y}$). Specifically,

for any period t in which no production occurs (i.e., $X_t^j = 0$) but the prediction indicates activation (i.e., $\hat{y}_t^j = 1$), the setup decision can be revised to $\hat{y}_t^j = 0$ to avoid paying the setup cost unnecessarily.

4.2 Instance generation

In order to generate the instances, we adopt the scheme proposed by Büyüktahtakın, Smith, and Hartman (2018). Two key parameters are used: the capacity-to-demand ratio, c, and the setup-to-holding cost ratio, f. Throughout the instance generation process, various parameters are sampled using a discrete uniform distribution, denoted as $U[a, b]$. Specifically, the shared capacity among items, c_t, is drawn from $U[0.8\, c\, \overline{d},\ 1.2\, c\, \overline{d}]$ where $\overline{d}$ represents the overall average demand. Additionally, the unit production cost p_t^j is sampled from $U[1, 200]$, the holding cost h_t^j from $U[1, 100]$, and the demand d_t^j from $U[500, 1500]$. The periodic setup cost f_t^j is determined from $U[0.9\, f\, \overline{h},\ 1.1\, f\, \overline{h}]$ with $\overline{h}$ denoting the overall average holding cost.

The experimental setup begins by generating a training set of $N = 12$ instances, each containing $\mathcal{J} = 8$ items, $\mathcal{T} = 60$ periods, and $c = 1.25 \cdot \mathcal{J}$. These instances are verified to be generated from the same distribution using the Anderson-Darling test (Anderson and Darling, 1954), ensuring that the model learns patterns that are representative of the data it will encounter on new instances. Then, to evaluate both the performance and scalability of the trained models, two additional sets of instances are produced. The first "easier" set employs $c = 1.25 \cdot \mathcal{J}$ and includes 30 instances each for $\mathcal{J} \in \{8,\ 16,\ 32\}$, totaling $N = 90$ easier instances*. Using $c = 1.25 \cdot \mathcal{J}$ yields instances with non-restrictive capacity constraints. This reduces the likelihood of the capacity constraint becoming active and, consequently, lowers competition among items for the shared resource. As a result, these instances are generally easier to solve than those with tighter capacity limits. Therefore, the second harder set uses $c = \mathcal{J}$ for $\mathcal{J} \in \{8,\ 16,\ 32\}$, thus also generating $N = 90$ instances. In all cases, the planning horizon is fixed at $T = 60$ periods, and the setup-to-holding cost ratio is set to $f = 1\,000$.

While all sets of instances are intended to be generated from the same distribution as the training set, only the easier instances with $\mathcal{J} = 8$ items were statistically validated to follow this distribution. The remaining instances could not be formally tested using the Anderson-Darling test due to computational limitations. Nonetheless, it remains important to evaluate the robustness of ML models under potential data distribution shifts.

To prevent excessively long runtimes on harder instances, we impose a solution time limit expressed in terms of work units, with a maximum of 150 units. Unlike time in seconds, work units provide a deterministic measure that is independent of specific hardware configurations or parallelization settings, ensuring consistent benchmarking. Additionally, we adopt Gurobi's default optimality gap tolerance of 0.01% for all experiments.

*It should be clarified that we did not extend to $\mathcal{J} = 64$ (or higher) despite the pattern of powers of two because initial tests showed that, at $c = 1.25 \cdot \mathcal{J}$, moving from 32 to 64 items produced negligible increases in solve time so to introduce greater difficulty without further scaling the model size, we instead tightened the capacity constraint to $c = \mathcal{J}$ on the same item sets.

A summary of work unit usage and optimality gaps for each set of instances is presented in Table 2. Note that reducing the capacity parameter c from 1.25 to 1.00 results in harder instances, as reflected by mean work units rising from below 8 to over 150, and a corresponding growth in optimality gaps.

Table 2: Work units and optimality gaps for training instances and for test instances with $c = 1.25, \mathcal{J}$ and $c = \mathcal{J}$ for $\mathcal{T} = 60$ and $\mathcal{J} = 8, 16, 32$

Use	Instance				Work unit			GAP optimal (%)		
	N	c	$\mathcal{T}$	$\mathcal{J}$	min	mean	max	min	mean	max
Train	12	1.25	60	8	0.215	0.630	1.205	0.000	0.003	0.009
Exp. I	30	1.25	60	8	0.288	0.692	1.047	0.000	0.004	0.010
	30	1.25	60	16	0.975	6.348	27.061	0.002	0.009	0.010
	30	1.25	60	32	1.169	7.610	20.792	0.007	0.009	0.010
Exp. II	30	1.00	60	8	12.724	111.347	150.314	0.000	0.073	0.359
	30	1.00	60	16	150.009	150.120	150.241	0.023	0.172	0.310
	30	1.00	60	32	150.014	150.176	151.347	0.067	0.151	0.224

4.3 Training dataset

The training process presented in Section 3 is applied using three different feature sets $\mathcal{H}$, as we rely on: (A) the parameters of the model, which are known in advance; (B) linear relaxation solutions, obtained by relaxing the binary variables to continuous values and solving the resulting linear program; and (C) a stronger linear relaxation solution , obtained by applying strengthening techniques (e.g., valid inequalities) during the cutting-plane phase of the branch-and-bound algorithm, with default cuts from the GUROBI solver. Additionally, we also evaluate models trained on (A+B) and (A+C) to investigate whether combining parameters with these linear relaxation solutions improves predictive performance.

To split the dataset, a value of $\alpha_1 = 0.75$ was used, resulting in $|\mathcal{D}_1| = 9$ training instances and $|\mathcal{D}_2| = 3$ test instances. A total of four different ML models m_λ were trained: K-Nearest Neighbors (KNN), Naïve Bayes (NB), DT, and SVM, with the addition of the ensemble learning model RF (i.e., $\lambda \in \Lambda := \{\text{KNN}, \text{NB}, \text{DT}, \text{SVM}, \text{RF}\}$). Grid search was implemented via random grid search (e.g., picking a random subset of hyperparameters) limited to a maximum of 40 combinations.

Table 3 provides a representative example using feature set (A), where the predicted binary decisions $\hat{y}_t^j$ are based on the input parameters of the MCLSP: demand d_t^j, fixed setup cost f_t^j, holding cost h_t^j, production cost p_t^j, and capacity c_t for each period t and item j. For instance, in instance $i = 1$ at period $t = 1$, item

Table 3: Example dataset for 30 MCLSP instances ($T = 60, J = 2$). Columns grouped as: $\mathcal{H}$ (Features: $d_t^j, f_t^j, h_t^j, p_t^j, c_t$), *Solver* (optimal solutions: $X_t^{*j}, Y_t^{*j}, I_t^{*j}$) and *ML model* (continuous prediction z, traditional discrimination rule prediction $z \geq \beta$ and proposed discrimination rule prediction $\hat{Y}_t^j$.

i	t	j	$\mathcal{H}^i$					Solver			ML model		
			d_t^j	f_t^j	h_t^j	p_t^j	c_t	X^{*j}_t	Y^{*j}_t	S^{*j}_t	z	$z \geq \beta$	$\hat{y}_t^j$
1	1	1	12	102	34	46	37	22	1	10	0.77	1	1
		2	15	91	57	37		15	1	0	0.64	1	1
	2	1	10	108	42	41	23	0	0	0	0.59	1	1
		2	13	115	38	40		13	1	0	0.34	0	1
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
30	60	1	41	24	61	89	0	0	0	0	0.42	0	1

$j = 1$, the model receives the inputs $d_t^j = 12$, $f_t^j = 102$, $h_t^j = 34$, $p_t^j = 46$, and $c_t = 37$. After training, the ML model produces a continuous prediction score $z = 0.77$. Applying the traditional discrimination rule with threshold $\beta = 0.5$ results in a classification of $\hat{y}t^j = 1$, which matches the outcome produced by our classification rule $F\beta(z)$. However, two types of misclassification can occur:

1. false positive: $z \geq \beta$ but the solver's decision Y_t^{*j} is 0 (e.g., instance $i = 1$ at $t = 2, j = 1$); this does not lead to infeasibility.

2. false negative: $z < \beta$ but production is required (e.g., instance $i = 1$ at $t = 2, j = 2$), which could render the solution infeasible.

Our proposed rule $F_\beta(z)$ then flips those false negatives to 1 in the last column ($\hat{y}_t^j$), thus restoring feasibility.

Training procedure resulted in a total of 30 models m_λ trained to predict the binary decision variable Y_t^{*j}. As reported in Table 4, the computational effort required to train these models is relatively consistent across all feature sets ($\mathcal{H}$) and machine learning algorithms, typically ranging between 500 and 550 seconds per model. In contrast, support vector machines (SVMs) exhibit significantly longer training times, from 650 up to 1,201 seconds, primarily due to the computational overhead associated with kernel-based hyperparameter tuning.

4.4 Results

This section presents the computational experiments, focusing on both solution quality and computational effort, measured in work units. The first set of experiments involves relatively easy instances, for which

Table 4: Training times for different ML models with various features.

Features	Time (s)				
	DT	KNN	NB	RF	SVM
A	527	527	529	541	1201
A+B	519	519	532	525	868
A+C	519	515	511	521	676
B	514	512	554	514	1551
C	512	509	509	514	650

optimal solutions are found within the imposed work unit limit. The second set includes harder instances, where achieving optimality is more challenging due to increased problem complexity.

First, we analyze the mean accuracy scores reported in Table 5 obtained from testing on $\mathcal{D}_2$ across different feature sets Λ, comparing the classical discrimination rule $z \geq \beta$ with the proposed discrimination rule $F_\beta(\boldsymbol{z})$. Since $F_\beta(\boldsymbol{z})$ adjusts the minimum number of predicted variables required to restore feasibility and leverages the continuous scores, it preserves the accuracy score with changes below 0.3%. In some cases, accuracy even improves. For example, employing KNN with feature set (A+C), the proposed discrimination rule unfixes variables that already had a high likelihood of being 1, leading to better alignment with the optimal solution.

Table 5: Mean accuracy scores before ($z \geq \beta$) and after ($F_\beta(z)$) applying the discrimination rule. The highest value of accuracy by the model was highlighted in bold.

Features	DT		KNN		NB		RF		SVM	
	$\overline{Acc}(z \geq \beta)$	$\overline{Acc}(\hat{y})$	$\overline{Acc}(z \geq \beta)$	$\overline{Acc}(\hat{y})$	$\overline{Acc}(z \geq \beta)$	$\overline{Acc}(\hat{y})$	$\overline{Acc}(z \geq \beta)$	$\overline{Acc}(\hat{y})$	$\overline{Acc}(z \geq \beta)$	$\overline{Acc}(\hat{y})$
A	0.737	0.738	0.676	0.674	0.777	0.775	0.784	0.785	0.694	0.694
A+B	0.815	0.814	0.843	0.845	0.793	0.800	0.821	0.819	0.835	0.835
A+C	**0.972**	**0.971**	**0.971**	**0.973**	**0.971**	**0.973**	0.971	0.972	0.969	**0.969**
B	0.814	0.812	0.811	0.810	0.763	0.768	0.828	0.833	0.812	0.814
C	**0.972**	0.970	0.962	0.960	0.968	0.968	**0.975**	**0.974**	**0.970**	**0.969**

ML models trained with feature set (A) exhibit reasonable predictive accuracy, which improves significantly when linear relaxation features (A+B and A+C) are included. The best performance is achieved, majority, with the feature set (A+C), which uses the stronger linear relaxation. Notably, feature sets that rely solely on linear relaxation based features (B and C) also perform competitively, often outperforming combined sets in several models, including GB, KNN, RF, and SVM.

Consequently, the selected ML model is RF, as it achieved the highest accuracy across the different feature sets. For the remaining analyses, we exclude the combined attribute sets and focus on evaluating RF-A, RF-B, and RF-C to assess model performance on both easy and hard instances.

4.4.1 Easy instances

We first assess the performance of the proposed matheuristic on easy instances, characterized by low computational resolution time. In addition, we examine the scalability of the models across instances with varying numbers of items.

Two metrics are introduced to evaluate the performance and the quality of the solution. Let $v(y^*)$, t^* denote the corresponding optimal objective function and runtime (work units) and $v(\hat{y})$, $\hat{t}$ denote the objective function and runtime (work units) of a solution obtained by the matheuristic. The computational time $\hat{t}$ includes both the time required to generate the features (e.g., solving the linear relaxation to obtain predictors B and C) and the time required to solve $F_\beta(\boldsymbol{z})$. The objective function gap and time gap are mathematically expressed, respectively, as:

$$\texttt{GapObj}(\%) = \frac{v(\hat{y}) - v(y^*)}{v(y^*)} \times 100, \quad \texttt{GapTime}(\%) = \frac{\hat{t} - t^*}{t^*} \times 100$$

Similarly, an additional metric is defined with respect to the linear relaxation. Let $v(y_{LR})$ denote the objective value of the linear relaxation solution y_{LR}. The integrality gap is computed as:

$$\texttt{GapInt}(\%) = \frac{v(y^*) - v(y_{LR})}{v(y^*)} \times 100$$

The computational time is measured in work units, and it is shown as t_{LR}.

Table 6 shows the performance obtained in the computational time and the qualities of the solution by our framework. The results show that when the number of items are 8 and 16, feature set (B) generally yields better performance than (A). For example, at 8 items, `GapObj` improves from a mean of 16.11% with (A) to 3.95% with (B), representing a relative improvement of approximately 75%. At 16 items, a similar trend is observed, with a `GapObj` reduction from 14.69% to 11.96%, corresponding to a 19% improvement. However, for the largest instance size (32 items), (A) becomes more effective than (B), likely because the simpler feature set avoids some overhead or inaccuracies that arise with larger problems in (B). Specifically, the mean `GapObj` for (A) is 12.48%, whereas for (B) it deteriorates significantly to 86.40%, representing a 7× increase in error. Similarly, in `GapTime`, (A) leads to a 65.53% reduction in runtime on average, compared to 55.26% for (B), suggesting (A) is also about 15.7% more efficient in computational time. Overall, the results demonstrate that incorporating valid inequalities and other enhancements into the relaxation (C) provides clear advantages in both computational time and solution quality as the problem scale increases. Moreover, using the framework can lead to improvements in runtime of up to almost 99%, while the objective function remains within less than 1% (in many cases, even below 0.5%) of the best solution found.

Table 6: Comparison of the minimum, mean, and maximum values for `GapTime(%)`, `GapObj(%)`, `GapInt(%)`, and `TimeLR` across different easy instance sizes (8, 16, and 32 items).

Items	Features	`GapTime(%)`			`GapObj(%)`			`GapInt(%)`			`TimeLR`		
		min	mean	max	min	mean	max	min	mean	max	min	mean	max
8	A	-85.58	-65.52	-0.72	7.56	16.11	25.75	-	-	-	-	-	-
8	B	-87.87	-77.60	-48.91	2.60	3.95	4.88	19.21	21.46	23.19	0.00	0.00	0.00
8	C	-88.39	-78.75	-55.51	0.13	0.39	0.89	0.18	0.27	0.42	0.10	0.12	0.14
16	A	-98.42	-81.94	-52.63	9.39	14.69	23.09	-	-	-	-	-	-
16	B	-98.11	-84.18	-53.80	9.43	11.96	13.71	21.12	22.76	24.07	0.01	0.01	0.01
16	C	-98.83	-89.80	-71.12	0.02	0.12	0.32	0.08	0.12	0.17	0.17	0.28	0.33
32	A	-93.32	-65.53	57.33	8.97	12.48	15.75	-	-	-	-	-	-
32	B	-91.09	-55.26	57.60	66.68	86.40	112.49	22.50	23.77	24.90	0.01	0.01	0.01
32	C	-95.32	-77.47	-22.29	0.02	0.06	0.13	0.02	0.04	0.05	0.45	0.92	1.11

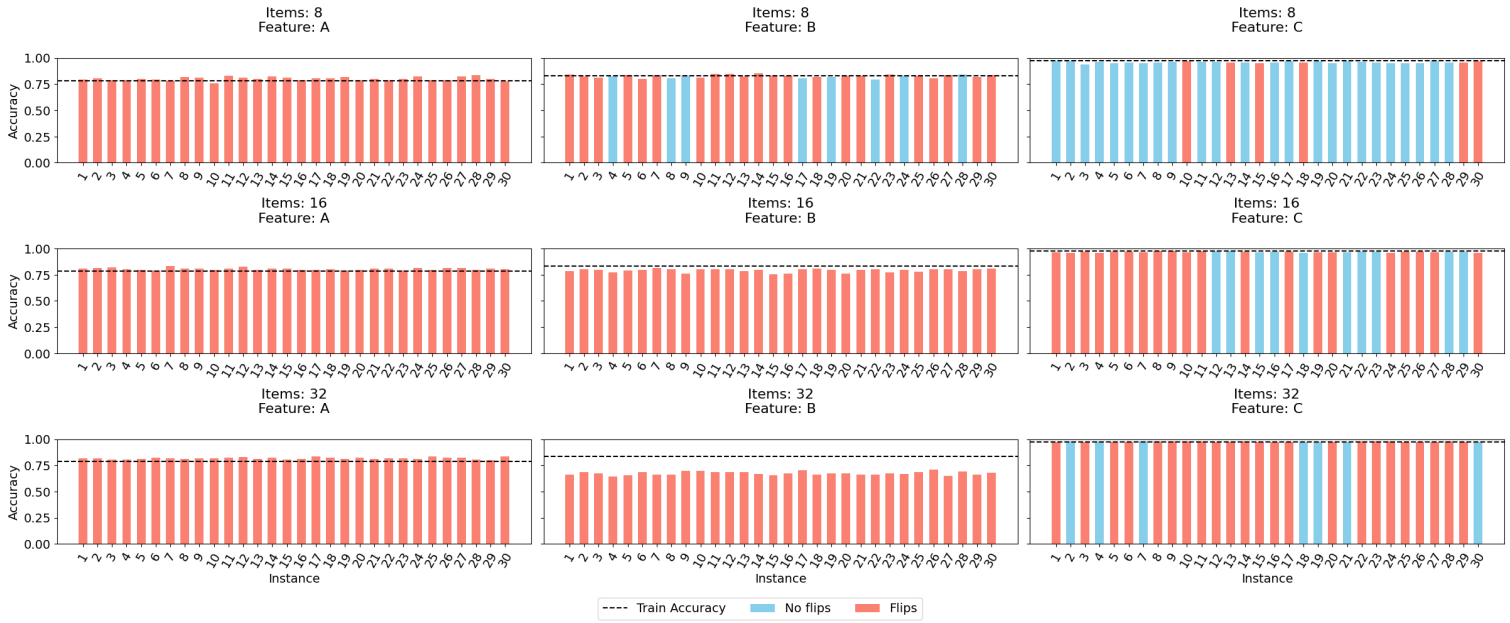


Figure 4: Accuracy per instance by item size and feature (A,B,C), including training accuracy (black dashed line). Orange bars indicate that at least one flip is realized from the discrimination rule $F_\beta(\boldsymbol{z})$, while blue bars indicate no flip is needed from the traditional discrimination rule.

Figure 4 evaluates the scalability of the ML models by examining how accuracy evolves with increasing problem size (i.e., number of items). Feature set (C) demonstrates a positive trend in average accuracy, rising from 0.96 (8 items) to 0.97 (16 items), and reaching 0.98 for 32 items. Feature set (A) maintains moderate stability, ranging from 0.80 to 0.82, while feature set (B) exhibits a significant decline in accuracy, decreasing from 0.83 (8 items) to 0.67 (32 items).

In addition, Figure 4 illustrates the utility of the proposed discrimination rule $F_\beta(\boldsymbol{z})$. Orange bars indicate instances where flips were required to recover feasibility, while blue bars denote instances that were feasible under the traditional discrimination rule. Feature set (A) shows a high number of flips across all items, suggesting limited predictive precision. Conversely, feature sets (B) and (C) often yield feasible predictions without requiring flips. Note that feature set (C) maintains a high number of feasible predictions even as instance size increases, although this number diminishes slightly for the largest (32-item) instances.

4.4.2 Hard instances

In contrast to the easy instances, where high test accuracy was observed, accuracy is not a meaningful metric for the harder instances, as they could not be solved to optimality. Instead, performance is evaluated using the `GapTime` and `GapObj` metrics, which capture improvements in solution quality and computational efficiency. These results are summarized in Table 7.

Table 7: Comparison of the minimum, mean, and maximum values for `GapTime(%)`, `GapObj(%)`, `GapInt(%)`, and `TimeLR` across different hard instance sizes (8, 16, and 32 items).

Items	Features	`GapTime(%)`			`GapObj(%)`			`GapInt(%)`			`TimeLR`		
		min	mean	max	min	mean	max	min	mean	max	min	mean	max
8	A	-99.66	-98.41	-90.56	4.01	8.61	31.67	-	-	-	-	-	-
8	B	-99.91	-99.74	-98.56	0.88	2.13	3.31	16.13	21.06	24.60	0.00	0.00	0.00
8	C	-99.91	-99.75	-98.75	0.08	0.67	1.69	0.78	1.04	1.46	0.11	0.14	0.17
16	A	-99.15	-99.08	-98.95	4.72	8.29	12.64	-	-	-	-	-	-
16	B	-99.15	-99.08	-98.96	4.47	7.63	11.55	21.18	23.65	26.97	0.01	0.01	0.01
16	C	-99.78	-99.68	-99.13	0.09	0.33	0.66	0.29	0.54	0.69	0.27	0.30	0.37
32	A	-98.83	-98.73	-98.60	6.86	9.39	14.69	-	-	-	-	-	-
32	B	-98.84	-98.76	-98.66	4.47	13.53	30.11	22.26	24.80	27.49	0.01	0.01	0.02
32	C	-99.45	-99.11	-98.65	0.14	0.24	0.40	0.15	0.23	0.31	0.73	0.85	1.00

For hard instances, across all feature sets (A, B, and C), the mean runtime required to obtain a feasible solution shows the most improvement for instances with 16 items, with an average `GapTime` reduction of

approximately −99.08%. For 8 and 32 items instances, the runtime improvements remain similar, except for feature set (B), where `GapTime` decreases from −99.74% (8 items) to −98.76% (32 items), indicating a relative loss of efficiency as the problem size increases.

Feature set (C) consistently provides the best overall performance in both `GapTime` and `GapObj`. For instance, while feature sets (B) and (C) achieve comparable `GapTime` values for 8-item instances (−99.74% and −99.75%, respectively), feature set (C) attains a lower mean `GapObj` of 0.67%, compared to 8.61% and 2.13% for feature sets (A) and (B), respectively. This advantage becomes more evident as the instance size increases: for 16 items instances, the mean `GapObj` achieved by feature set (C) is 0.33%, compared to 8.29% and 7.63% for feature sets (A) and (B), respectively. Similarly, with 32 item instances, feature set (C) yields a mean `GapObj` of just 0.24%, while sets (A) and (B) result in 9.39% and 13.53%, respectively.

Notably, although the stronger linear relaxation (feature set C) incurs greater computational cost than using the linear relaxation (feature set B), the associated reduction in the integrality gap (`GapInt`) appears to contribute positively to the quality of the resulting predictions. This observation may help explain the comparatively stronger performance of models trained with feature set (C).

5 Conclusion

This paper proposed a matheuristic to solve combinatorial optimization (CO) problems using a predict-then-optimize framework that balances predictive confidence with solution feasibility. The proposed framework addresses three fundamental challenges in the predict-then-optimize paradigm: feasibility, modelling, and scalability.

First, regarding feasibility, traditional threshold-based classification does not guarantee feasible solutions when ML-predicted binary variables are fixed in the MILP. To address this, we proposed a discrimination rule inspired by Phase I of the Simplex method. This mechanism adjusts infeasible predictions to ensure feasibility while minimizing the number of variable flips, thereby preserving prediction quality. Results show that as prediction accuracy increases, fewer variable flips are required, reinforcing the value of accurate ML models in CO contexts.

Second, from a modelling perspective, we evaluated the contribution of different feature sets. Features derived from model parameters (feature set A) and linear relaxations (feature sets B and C) showed that stronger relaxations significantly enhance ML model accuracy. Additionally, the discrimination rule benefits from the binary structure of MILPs, enabling efficient feasibility recovery.

Third, the framework demonstrated strong scalability, even on data drift. ML models trained on small instances (8 items) generalize well to larger ones (32 items) when using feature sets (A) and (C). Feature set (B), in contrast, requires stronger relaxations to maintain predictive quality at scale.

Finally, experimental results on the Multi-item Capacitated Lot-Sizing Problem (MCLSP) demonstrated

computational time reductions of up to 99% compared to a commercial solver (GUROBI), while maintaining optimality gaps consistently below 1% across varying instance sizes and complexities. Thus the proposed approach is shown to be competitive in terms of solution quality, computation time, and scalability compared to methods in the existing literature (e.g., (Yilmaz and Büyüktahtakın, 2024)). Unlike the artificial neural network, it requires significantly fewer solved instances for training and consumes fewer computational resources, making it attractive for real-world settings where computing infrastructure is limited. This is especially relevant in operational environments where repetitive, time-sensitive decisions must be made and deep learning methods may be impractical to implement.

Future research could further investigate the scalability limits of the proposed ML model by evaluating how effectively models trained on smaller instances generalize to larger problems in terms of item quantity and planning periods. Additionally, assessing the transferability of this framework to other MILP problems could highlight its broader applicability and potential limitations. Another promising avenue involves using predicted scores z, understanding them as probabilities, to guide branching decisions within the search tree, potentially transforming predictions into guiding for optimal solutions. Moreover, extending the framework to operate at each node of the branch and bound tree by generating new candidate solutions based on the linear relaxation at that node could further enhance search efficiency and improve overall solution quality. Finally, by relying on inherently interpretable techniques, we not only alleviate the "black-box" concerns of deep learning but also lay the groundwork for future studies to analyze feature importance and uncover exactly which problem attributes drive high-accuracy predictions. Exploring these dimensions will enhance the practicality and broader applicability of ML model-integrated optimization methods.

References

Alvarez, A. M., Louveaux, Q., and Wehenkel, L. (2014). "A Supervised Machine Learning Approach to Variable Branching in Branch-And-Bound."

Anderson, T. W. and Darling, D. A. (1954). "A Test of Goodness of Fit." *Journal of the American Statistical Association* 49.268, pp. 765–769. DOI: https://doi.org/10.1080/01621459.1954.10501232.

Archetti, C. and Speranza, M. G. (2014). "A survey on matheuristics for routing problems." *EURO Journal on Computational Optimization* 2.4, pp. 223–246. DOI: https://doi.org/10.1007/s13675-014-0030-7.

Bello, I. et al. (2016). "Neural combinatorial optimization with reinforcement learning." DOI: https://doi.org/10.48550/arXiv.1611.09940.

Bengio, Y., Lodi, A., and Prouvost, A. (2021). "Machine learning for combinatorial optimization: a methodological tour d'horizon." *European Journal of Operational Research* 290.2, pp. 405–421. DOI: https://doi.org/10.1016/j.ejor.2020.07.063.

Berrar, D. (2019). "Cross-Validation." *Encyclopedia of Bioinformatics and Computational Biology*. Ed. by S. Ranganathan et al. Oxford: Academic Press, pp. 542–545. DOI: https://doi.org/10.1016/B978-0-12-809633-8.20349-X.

Bertsimas, D. and Tsitsiklis, J. N. (1997). *Introduction to Linear Optimization*. Belmont, MA: Athena Scientific.

Bertsimas, D. and Weismantel, R. (2005). *Optimization over integers*. Athena Scientific, pp. I–X, 1–602.

Bischl, B. et al. (2016). "Aslib: A benchmark library for algorithm selection." *CoRR* 237, pp. 41–58. DOI: https://doi.org/10.48550/arXiv.1506.02465.

Burke, E. K. et al. (2019). "A classification of hyper-heuristic approaches: revisited." *Handbook of meta-heuristics*, pp. 453–477. DOI: https://doi.org/10.1007/978-3-319-91086-4_14.

Büyüktahtakın, . E., Smith, J. C., and Hartman, J. C. (2018). "Partial objective inequalities for the multi-item capacitated lot-sizing problem." *Computers & Operations Research* 91, pp. 132–144. DOI: https://doi.org/10.1016/j.cor.2017.11.006.

Chen, T. et al. (2022). "Learning to optimize: A primer and a benchmark." *Journal of Machine Learning Research* 23.189, pp. 1–59. DOI: https://doi.org/10.48550/arXiv.2103.12828.

Cornuéjols, G. (2008). "Valid inequalities for mixed integer linear programs." *Mathematical programming* 112.1, pp. 3–44. DOI: https://doi.org/10.1007/s10107-006-0086-0.

Cunha, J. O., Kramer, H. H., and Melo, R. A. (2019). "Effective matheuristics for the multi-item capacitated lot-sizing problem with remanufacturing." *Computers & Operations Research* 104, pp. 149–158. DOI: https://doi.org/10.1016/j.cor.2018.12.012.

Daskin, M. S. and Maass, K. L. (2015). "The p-Median Problem." *Location Science*. Ed. by G. Laporte, S. Nickel, and F. Saldanha da Gama. Cham: Springer International Publishing, pp. 21–45. DOI: https://doi.org/10.1007/978-3-319-13111-5_2.

Desale, S. et al. (2015). "Heuristic and meta-heuristic algorithms and their relevance to the real world: a survey." *Int. J. Comput. Eng. Res. Trends* 351.5, pp. 2349–7084.

Desaulniers, G., Desrosiers, J., and Solomon, M. M. (2006). *Column generation*. Vol. 5. Springer Science & Business Media. DOI: https://doi.org/10.1007/b135457.

Doroudi, S. (2020). "The Bias-Variance Tradeoff: How Data Science Can Inform Educational Debates." *AERA Open* 6.4, p. 2332858420977208. DOI: https://doi.org/10.1177/2332858420977208.

Fan, F.-L. et al. (2021). "On Interpretability of Artificial Neural Networks: A Survey." *IEEE Transactions on Radiation and Plasma Medical Sciences* 5.6, pp. 741–760. DOI: https://doi.org/10.1109/TRPMS.2021.3066428.

Feurer, M. and Hutter, F. (2019). "Hyperparameter Optimization." *Automated Machine Learning: Methods, Systems, Challenges*. Ed. by F. Hutter, L. Kotthoff, and J. Vanschoren. Cham: Springer International Publishing, pp. 3–33. DOI: https://doi.org/10.1007/978-3-030-05318-5_1.

Gnägi, M. and Baumann, P. (2021). "A matheuristic for large-scale capacitated clustering." *Computers & operations research* 132, p. 105304. DOI: https://doi.org/10.1016/j.cor.2021.105304.

Hastie, T. et al. (2005). "The elements of statistical learning: data mining, inference and prediction." *The Mathematical intelligencer* 27.2, pp. 83–85. DOI: https://doi.org/10.1007/978-0-387-84858-7.

Hoffman, K. L. (2000). "Combinatorial optimization: Current successes and directions for the future." *Journal of Computational and Applied Mathematics* 124.1. Numerical Analysis 2000. Vol. IV: Optimization and Nonlinear Equations, pp. 341–360. DOI: https://doi.org/10.1016/S0377-0427(00)00430-1.

Hussein, A. et al. (2017). "Imitation learning: A survey of learning methods." *ACM Computing Surveys (CSUR)* 50.2, pp. 1–35. DOI: https://doi.org/10.1145/3054912.

Hutter, F. et al. (2014). "AClib: A benchmark library for algorithm configuration." *Learning and Intelligent Optimization: 8th International Conference, Lion 8, Gainesville, FL, USA, February 16-21, 2014. Revised Selected Papers 8*. Springer, pp. 36–40. DOI: https://doi.org/10.1007/978-3-319-09584-4_4.

Jans, R. and and, Z. D. (2008). "Modeling industrial lot sizing problems: a review." *International Journal of Production Research* 46.6, pp. 1619–1643. DOI: 10.1080/00207540600902262.

Jia, H. and Shen, S. (June 2019). "Benders Cut Classification via Support Vector Machines for Solving Two-stage Stochastic Programs." *arXiv*. DOI: https://doi.org/10.48550/arXiv.1906.05994.

Karimi, B., Fatemi Ghomi, S., and Wilson, J. (2003). "The capacitated lot sizing problem: a review of models and algorithms." *Omega* 31.5, pp. 365–378. DOI: https://doi.org/10.1016/S0305-0483(03)00059-8.

Khalil, E. et al. (2016). "Learning to branch in mixed integer programming." *Proceedings of the AAAI conference on artificial intelligence*. Vol. 30. 1.

Khalil, E. B. et al. (2017). "Learning to Run Heuristics in Tree Search." *Ijcai*, pp. 659–666. DOI: https://doi.org/10.24963/ijcai.2017/92.

Kotary, J. et al. (2021). "End-to-end constrained optimization learning: A survey." *CoRR*. DOI: https://doi.org/10.48550/arXiv.2103.16378.

Larsen, E. et al. (2022). "Predicting tactical solutions to operational planning problems under imperfect information." *INFORMS Journal on Computing* 34.1, pp. 227–242. DOI: https://doi.org/10.1287/ijoc.2021.1091.

Lin, X. et al. (2019). "Approximate Mixed-Integer Programming Solution with Machine Learning Technique and Linear Programming Relaxation." *2019 3rd International Conference on Smart Grid and Smart Cities (ICSGSC)*, pp. 101–107. DOI: https://doi.org/10.1109/ICSGSC.2019.00-11.

Mart, R., Pardalos, P. M., and Resende, M. G. (2018). *Handbook of heuristics*. Springer Publishing Company, Incorporated. DOI: https://doi.org/10.1007/978-3-319-07124-4.

Morabit, M., Desaulniers, G., and Lodi, A. (2022). *Learning to repeatedly solve routing problems*. DOI: https://doi.org/10.1002/net.22200.

Morabit, M., Desaulniers, G., and Lodi, A. (2023). "Machine-Learning–Based Arc Selection for Constrained Shortest Path Problems in Column Generation." *INFORMS Journal on Optimization* 5.2, pp. 191–210. DOI: https://doi.org/10.1287/ijoo.2022.0082.

Nair, V. et al. (2020). "Solving mixed integer programs using neural networks." DOI: https://doi.org/10.48550/arXiv.2012.13349.

Rachelson, E., Abbes, A. B., and Diemer, S. (Nov. 2010). "Combining Mixed Integer Programming and Supervised Learning for Fast Re-planning." 2, pp. 63–70. DOI: https://doi.org/10.1109/ICTAI.2010.85.

Rahmaniani, R. et al. (2017). "The Benders decomposition algorithm: A literature review." *European Journal of Operational Research* 259.3, pp. 801–817. DOI: https://doi.org/10.1016/j.ejor.2016.12.005.

Reeves, C. R. (1996). "Heuristic search methods: A review." *Operational Research*, pp. 122–149.

Sultana, N. et al. (Aug. 2022). "Learning to optimise general TSP instances." *Int. J. Mach. Learn. Cybern.* 13.8, pp. 2213–2228. DOI: https://doi.org/10.1007/s13042-022-01516-8.

Tsimenidis, S. (2020). *Limitations of Deep Neural Networks: a discussion of G. Marcus' critical appraisal of deep learning*. DOI: https://doi.org/10.48550/arXiv.2012.15754.

Vanderbei, R. J. (2020). *Linear Programming: Foundations and Extensions*. 5th ed. New York: Springer. DOI: https://doi.org/10.1007/978-3-030-39415-8.

Vanderschueren, T. et al. (2022). "Predict-then-optimize or predict-and-optimize? An empirical evaluation of cost-sensitive learning strategies." *Information Sciences* 594, pp. 400–415. DOI: https://doi.org/10.1016/j.ins.2022.02.021.

Xavier, A. S., Qiu, F., and Ahmed, S. (2019). *Learning to Solve Large-Scale Security-Constrained Unit Commitment Problems*. DOI: https://doi.org/10.48550/arXiv.1902.01697.

Yilmaz, D. and Büyüktahtakın, İ. E. (July 2022). "Learning Optimal Solutions via an LSTM-Optimization Framework." *arXiv*. DOI: https://doi.org/10.48550/arXiv.2207.02937.

Yilmaz, D. and Büyüktahtakın, İ. E. (Apr. 2024). "An expandable machine learning-optimization framework to sequential decision-making." *Eur. J. Oper. Res.* 314.1, pp. 280–296. DOI: https://doi.org/10.1016/j.ejor.2023.10.045.

Zhang, C. et al. (2022). "Exact and matheuristic methods for the parallel machine scheduling and location problem with delivery time and due date." *Computers & Operations Research* 147, p. 105936. DOI: https://doi.org/10.1016/j.cor.2022.105936.

A Hyperparameters

Table 8: Grid of hyperparameters evaluated for each model

Model	Hyperparameters and values evaluated
SVM	C: [0.1, 1, 10, 100, 1000]
	gamma: [10, 1, 0.1, 0.01, 0.001]
	kernel: {rbf, poly}
	degree (poly only): [0, 1, 2, 3]
KNN	algorithm: {auto, ball_tree}
	metric: {minkowski, chebyshev}
	n_neighbors: {1, 3, 5, …, 49}
	p: {1, 2}
	weights: {uniform, distance}
RF	n_estimators: [5, 10, 12]
	max_features: {None, sqrt, log2}
	max_depth: [2, 3, 5, 10]
	min_samples_split: [3]
	min_samples_leaf: [10, 20, 30, 40]
	bootstrap: {True, False}
DT	criterion: {gini, entropy}
	splitter: {best, random}
	max_depth: [2, 3, 5, 10]
	min_samples_split: [3]
	min_samples_leaf: [10, 20, 30, 40]
	min_weight_fraction_leaf: [0.0, 0.1, 0.2]
	max_features: {sqrt, log2, 0.7}
	max_leaf_nodes: [20, 50, 100]
	min_impurity_decrease: [0.0, 0.05, 0.1]
	ccp_alpha: [0.0, 0.001, 0.01, 0.1]
NB	var_smoothing: logspace(-12, 1, 150)

Table 9: The best hyperparameters of ML models selected

Feature	Model	best hyperparameters	μ_{Acc}	σ_{Acc}	CV=$\frac{\sigma_{\text{Acc}}}{\mu_{\text{Acc}}}$
A	RF	'n_estimators': 5 'min_samples_split': 3 'min_samples_leaf': 20 'max_features': 'sqrt' 'max_depth': 5 'bootstrap': False	0.786	0.02	0.024
B	RF	'n_estimators': 5 'min_samples_split': 3 'min_samples_leaf': 20 'max_features': 'sqrt' 'max_depth': 3 'bootstrap': False	0.828	0.02	0.017
C	RF	'n_estimators': 12 'min_samples_split': 3 'min_samples_leaf': 40 'max_features': 'log2' 'max_depth': 10 'bootstrap': True	0.980	0.01	0.009